UNIT I: Principles of Object Oriented Programming & Control Structures

Question Bank Answers: Debugging & Pseudo-Code Formulation

Name	Nikitha R
Reg. No	19252421
Course Code & Name	DSA0105 Object Oriented Programming with C++

Contents

- **Section A: Debugging** (5 problems): each answer gives the line number and error classification, root cause, corrected code, and the dry-run with exact output.
- **Section B: Pseudo-Code** (5 problems): each answer gives a design overview, structured pseudo-code, and a trace table.
- All corrected C++ programs were compiled with g++ (`-Wall`) and run, and the outputs shown are the real outputs. Line numbers follow the numbering used in the question paper (code lines counted from the first line of each listing; where the paper's comments name a line, that line is used).

SECTION A: DEBUGGING QUESTIONS

Question 1: Operator Precedence Trap with Bitwise Operations & Implicit Promotion

Topic: Operators in C++, Operator Precedence, Implicit Conversion

Step 1: Analyse the problem statement

- **Expected:** print "Valid Odd" only when the 8-bit value is *odd* AND *greater than 10*; count down 3, 2, 1, 0 and stop.
- **Actual:** the odd test is written with a precedence trap, and the loop condition can never become false.

Step 2: Errors found

Line	Classification	Defect
Line 7 region: `if (num & 1 == 1 && num > 10)`	Logic Defect (latent precedence bug)	== binds tighter than &, so the test is parsed as `num & (1 == 1)`.
Line 14: `for (unsigned char i = 3; i >= 0; --i)`	Logic Defect / infinite loop	An unsigned type is never negative, so `i >= 0` is always true.

Step 3: Root cause explanation

- **Precedence (Line 7).** In C++ the relational/equality operators (==) have *higher* precedence than the bitwise AND (&). So `num & 1 == 1` is evaluated as `num & (1 == 1)`. The sub-expression `1 == 1` yields `true`, which is promoted to `int 1`, giving `num & 1`. The unsigned char `num` is also integer-promoted to `int` before the & is applied.
- For the mask value 1 this happens to give the right answer, **which is exactly why it is a trap**. For any other mask the result is wrong: `num & 4 == 4` becomes `num & 1`, which tests bit 0 instead of bit 2 (confirmed by compiling: for `num = 4` the test is false even though bit 2 is set). GCC warns with `-Wparentheses`: "suggest parentheses around comparison in operand of '&'".
- **Loop (Line 14).** `unsigned char` holds 0..255. After `i` reaches 0, `--i` wraps modulo 256 to 255, and `255 >= 0` is still true, so the loop never ends. The student's `if (i == 0) break;` is only a patch: remove it and the loop is infinite. Also, `std::cout << i` for a `unsigned char` would print a character, hence the `(int)` cast.

Step 4: Corrected code

```
#include <iostream>

int main() {
    unsigned char num = 15;

    // Parentheses force the bitwise AND to be evaluated first
    if ((num & 1) == 1 && num > 10) {
        std::cout << "Valid Odd Number: " << (int)num << std::endl;
    } else {
        std::cout << "Condition Failed for: " << (int)num << std::endl;
    }

    // Use a signed int counter so that i >= 0 can become false
    for (int i = 3; i >= 0; --i) {
        std::cout << i << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

Step 5: Dry run & predicted output

Step	Evaluation	Result
1	num = 15 (binary 0000 1111); `(num & 1)` = 0000 1111 & 0000 0001 = 1	1 == 1 is true
2	num > 10 : 15 > 10	true
3	true && true	prints "Valid Odd Number: 15"
4	i = 3: 3 >= 0 true, print 3, i = 2	prints 3
5	i = 2, then i = 1, then i = 0: each prints and decrements	prints 2, 1, 0
6	i = -1: -1 >= 0 false	loop ends

```
Valid Odd Number: 15
3 2 1 0
```

Question 2: Scope Resolution & Variable Shadowing Conflicts

Topic: Variables, Scope Resolution Operator (`::`), Constants

Step 1: Analyse the problem statement

- **Expected:** compute one total with the global 18% rate and one with the local 5% rate.
- **Actual:** a typo stops compilation, and the "local" total silently uses the global rate.

Step 2: Errors found

Line	Classification	Defect
Line 11: `itemPrice * :taxRate`	Compilation Error (syntax)	A single colon is not an operator in an expression. The scope resolution operator is the two-character token `::`. GCC: "expected primary-expression before ':' token".
Line 16: `itemPrice * ::taxRate`	Logic Defect (semantic)	Compiles, but `::taxRate` is the **global** 0.18, not the enclosing local 0.05.

Step 3: Root cause explanation

- **Line 11.** The lexer produces the token `:` (used in `?:`, labels, base-class lists) which cannot start an operand. Writing `::taxRate` (unary scope resolution) is required.
- **Line 16.** The unary `::` always looks the name up in the **global namespace scope** only. It can **not** reach an enclosing *function-local* scope. So even though the inner block has an `int taxRate = 12` and the outer function scope has `double taxRate = 0.05`, the expression `::taxRate` resolves to the global `0.18`. Inner total would therefore be 1000 + 1000 × 0.18 = **1180** instead of the intended 1050.
- **Shadowing rules.** A name declared in an inner block hides (shadows) the same name from enclosing blocks and from the global scope, from the point of its declaration to the end of that block. The hidden outer *local* variable has no qualified syntax to reach it. The only fix is to copy it into a differently named variable *before* shadowing, or to use distinct names. Note also that `int taxRate = 12` would truncate any fractional rate, so a separate, clearly named integer is the cleaner design.

Step 4: Corrected code

```
#include <iostream>

double taxRate = 0.18;                  // Global standard tax rate (18%)

int main() {
    double itemPrice = 1000.0;
    double taxRate = 0.05;              // Local subsidised rate (5%) shadows the global

    std::cout << "Local Rate: " << taxRate << std::endl;

    // Line 11 fixed: '::' (two colons) reaches the global variable
    double totalWithGlobalTax = itemPrice + (itemPrice * ::taxRate);

    {
        // Capture the outer local rate BEFORE giving the inner block its own names
        double outerLocalRate = taxRate;
        int corporateTaxRate = 12;      // distinct name, no more shadowing

        double totalWithLocalTax = itemPrice + (itemPrice * outerLocalRate);
        std::cout << "Inner total: " << totalWithLocalTax << std::endl;
        std::cout << "Corporate tax rate: " << corporateTaxRate << "%" << std::endl;
    }

    std::cout << "Final Global Calculation: " << totalWithGlobalTax << std::endl;
    return 0;
}
```

Step 5: Dry run & predicted output

Statement	Value computation	Result
Local Rate print	local taxRate	0.05
totalWithGlobalTax	1000 + 1000 × 0.18 (global)	1180
outerLocalRate	copy of local taxRate	0.05
totalWithLocalTax	1000 + 1000 × 0.05	1050
corporateTaxRate	inner int	12

```
Local Rate: 0.05
Inner total: 1050
Corporate tax rate: 12%
Final Global Calculation: 1180
```

Question 3: Switch-Case Variable Initialization & Illegal Case Types

Topic: Control Structures (`switch-case`), Tokens, Data Types

Step 1: Analyse the problem statement

- **Expected:** for `tokenType == 2` the value should be 20 × 5 = 100, then stop.
- **Actual:** the compiler rejects the program; and, even if it compiled, case 2 would fall through into case 3.

Step 2: Errors found

Location	Classification	Defect
`int offset = 10;` / `int multiplier = 5;` declared directly under case labels	Compilation Error	"jump to case label crosses initialization". Reported at `case 2`, `case 3`, `case 4.5` and `default`.
`case 4.5:`	Compilation Error	Case label must be an integral (or enum) constant expression.
Missing `break` at end of case 2	Logic Defect	Control falls through into case 3.

Step 3: Root cause explanation

- **Cross-initialization.** All case labels share *one* block scope (the braces of the switch). A variable declared with an initializer under `case 1` is in scope at `case 2`, `case 3` and `default`, but a jump to those labels bypasses the initialization. C++ forbids a jump that enters the scope of a variable past its initialization (for a variable with a non-trivial or initialized declaration), so the compiler flags every later label. The same applies to `multiplier` when jumping to case 3/default. **Fix:** wrap each case body in its own braces `{ ... }` so the variable's scope ends before the next label (or declare it before the switch).
- **Illegal case label.** The switch condition and every `case` expression must be of integral or enumeration type (`int, char, short, long, bool, enum`, and their unsigned forms), and each case label must be a *constant expression* that converts to the condition type without narrowing. `float`/`double`, strings and other class types are not permitted. `4.5` is a floating literal, so it is rejected (its conversion to int would also be narrowing). The label must be an integer such as `4`.
- **Fall-through.** Without `break`, execution continues into the next label. For `tokenType == 2`: tokenValue = 100, then case 3 adds 100, giving **200** instead of 100. This is a logic defect, not a compile error.

Step 4: Corrected code

```
#include <iostream>

int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;

    switch (tokenType) {
        case 1: {                       // own scope: offset dies at the closing brace
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }
        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;                      // prevents fall-through into case 3
        }
        case 3:
            tokenValue += 100.0f;
            break;
        case 4:                         // integral constant, not 4.5
            std::cout << "Float token detected" << std::endl;
            break;
        default:
            std::cout << "Unknown token" << std::endl;
    }

    std::cout << "Calculated Value: " << tokenValue << std::endl;
    return 0;
}
```

Step 5: Dry run & predicted output

Step	Action	State
1	tokenType = 2, tokenValue = 0	switch jumps to `case 2`
2	multiplier = 5; tokenValue = 20.0 × 5	tokenValue = 100
3	`break` leaves the switch	case 3 is skipped
4	print tokenValue	100

```
Calculated Value: 100
```

Question 4: Reference Variable Initialization & Constant Binding Violations

Topic: Reference Variables, Constants, Implicit Conversions

Step 1: Analyse the problem statement

- **Expected:** references act as aliases for a, a literal, a converted value and a constant.
- **Actual:** four separate reference rules are violated, so compilation fails (all four are reported by g++).

Step 2: Errors found (Student Task 1)

Line	Code	Classification	Rule violated
Line 7	`int &refA;`	Compilation Error	A reference must be initialized at its declaration; it cannot be left unbound and assigned later (`refA = a` would assign through it, not bind it). GCC: "'refA' declared as reference but not initialized".
Line 11	`int &refLiteral = 100;`	Compilation Error	A non-const lvalue reference cannot bind to an rvalue (the literal 100 has no address/identity).
Line 14	`int &refInt = pi;`	Compilation Error / Type Error	`pi` is a `double`. Binding to `int&` needs a double-to-int conversion that creates a *temporary* rvalue, which a non-const lvalue reference cannot bind to.
Line 18 (the assignment after `constRef`)	`constRef = constRef + 10;`	Compilation Error	A reference to const gives read-only access. Assigning through it is ill-formed: "assignment of read-only reference".

Step 3: Lifetime extension of temporaries (Student Task 2)

A `const T&` (or an rvalue reference) *may* bind to a temporary. When it does, the temporary's lifetime is **extended to the lifetime of the reference** instead of ending at the end of the full-expression. Example: `const int &r = 50;` makes the compiler create an anonymous `int` holding 50, bind `r` to it, and keep that object alive until `r` goes out of scope. Likewise `const int &x = pi;` converts 3.14159 to 3 into a temporary and binds to that temporary, so `x` **is NOT an alias of** `pi` and later changes to `pi` are not seen. Non-const references are denied this because modifying a disposable temporary is almost always a bug.

Step 4: Corrected code (Student Task 3)

```
#include <iostream>

int main() {
    int a = 25;

    int &refA = a;                    // bound at declaration: true alias of a

    const int &refLiteral = 100;      // const ref may bind to an rvalue (lifetime extended)

    double pi = 3.14159;
    const int &refInt = pi;           // binds to a temporary int(3); NOT an alias of pi

    const int &constRef = 50;
    int result = constRef + 10;       // read the const ref, store the result in a normal int

    std::cout << a << " " << refA << std::endl;
    std::cout << refLiteral << " " << refInt << " " << constRef << " " << result << std::endl;
    return 0;
}
```

Step 5: Dry run & predicted output (Student Task 4)

Statement	Effect	Value
`int &refA = a;`	refA aliases a	a = refA = 25
`const int &refLiteral = 100;`	temp int 100, lifetime extended	100
`const int &refInt = pi;`	3.14159 truncated to int temp	3
`const int &constRef = 50;`	temp int 50	50
`result = constRef + 10`	50 + 10	60

```
25 25
100 3 50 60
```

Question 5: Dynamic Array Allocation & Stray Loop Semicolon

Topic: Memory Operators (`new`, `delete`), Loops, Off-by-One Boundaries

Step 1: Analyse the problem statement

- **Expected:** fill five readings 10, 20, 30, 40, 50, sum them (150) and free the heap block correctly.
- **Actual:** the stray semicolon breaks the initialization loop (and compilation), and the deallocation is mismatched.

Step 2: Errors found

Line	Classification	Defect
Line 10: `for (...);`	Compilation Error (and, if `i` existed outside, a Runtime Bug)	The `;` is the whole loop body (an empty statement). The braces that follow are a plain block executed once, outside the loop.
Line 21: `delete readings;`	Undefined Behavior	Memory obtained with `new[]` must be released with `delete[]`.

Step 3: Root cause explanation

- **Stray semicolon (Task 1).** `for (int i = 0; i < size; ++i);` loops five times doing nothing. The counter `i` is declared in the for-init-statement, so its scope is only the for statement. In the following block `{ readings[i] = (i + 1) * 10; }` the name `i` is undeclared, so GCC reports "'i' was not declared in this scope". If an outer `i` did exist (say after the loop it would equal 5), the block would run once and write `readings[5]`, which is one element past the end of the 5-element heap array: a **heap buffer overflow** (undefined behavior), while elements 0-4 stay uninitialized and the sum is garbage.
- **delete vs delete[] (Task 2).** `delete p` destroys a single object allocated by plain `new`. `delete[] p` destroys an array allocated by `new[]`: it runs the destructor of *every* element and uses the stored element count/bookkeeping to free the whole block. Using `delete` on an array is undefined behavior (it may free the wrong address, leak, corrupt the heap, or skip destructors for class types), even though it often appears to work for plain `int`.

Step 4: Corrected code (Task 3)

```
#include <iostream>

int main() {
    int size = 5;
    int *readings = new int[size];            // allocate 5 ints on the heap

    for (int i = 0; i < size; ++i) {          // no semicolon after the for header
        readings[i] = (i + 1) * 10;
    }

    int total = 0;
    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }
    std::cout << "Total: " << total << std::endl;

    delete[] readings;                        // matches new[]
    readings = nullptr;                       // avoid a dangling pointer
    return 0;
}
```

Step 5: Trace & final output (Task 4)

i	readings[i] = (i + 1) * 10	total after adding
0	10	10
1	20	30
2	30	60
3	40	100
4	50	150

```
Total: 150
```

SECTION B: PSEUDO-CODE WRITING QUESTIONS

Question 1: Procedural-to-OOP Paradigm Transformation

Topic: OOP Paradigm, Encapsulation, Data Hiding, Abstraction

Design overview

- **Encapsulation / data hiding:** all five attributes are PRIVATE; the outside world can only change them through validated public methods.
- **Abstraction:** Main only calls Initialize, CalculateConsumption, ComputeBill and DisplayBill and never needs to know how tiers are computed.
- **Validation:** Initialize rejects negative readings and current < previous, setting a BOOLEAN `isValid` flag; the other methods refuse to run on invalid objects.
- **Tiered tariff** using nested selection: 100 units at 1.50; next 200 at 2.50; remainder at 4.00 plus 10% surcharge on the *entire* bill only when units exceed 300.

Structured pseudo-code

```
CLASS WaterConsumer
  PRIVATE:
    STRING  consumerID
    REAL    previousReading
    REAL    currentReading
    REAL    unitsConsumed
    REAL    totalBill
    BOOLEAN isValid

  PUBLIC:
    METHOD Initialize(STRING id, REAL prev, REAL curr)
      isValid <- TRUE
      IF prev < 0 OR curr < 0 THEN
        isValid <- FALSE
        DISPLAY "ERROR: readings must be non-negative"
      ELSE
        IF curr < prev THEN
          isValid <- FALSE
          DISPLAY "ERROR: current reading is less than previous reading"
        ENDIF
      ENDIF
      IF isValid = TRUE THEN
        consumerID      <- id
        previousReading <- prev
        currentReading  <- curr
        unitsConsumed   <- 0.0
        totalBill       <- 0.0
      ENDIF
    ENDMETHOD

    METHOD CalculateConsumption()
      IF isValid = TRUE THEN
        unitsConsumed <- currentReading - previousReading
      ENDIF
    ENDMETHOD

    METHOD ComputeBill()
      REAL tier1Units, tier2Units, tier3Units
      REAL tier1Cost,  tier2Cost,  tier3Cost
      REAL baseBill, surcharge
      IF isValid = FALSE THEN
        RETURN
      ENDIF
      tier1Units <- 0.0 ; tier2Units <- 0.0 ; tier3Units <- 0.0
      IF unitsConsumed <= 100 THEN
        tier1Units <- unitsConsumed
      ELSE
        tier1Units <- 100
```

```
        IF unitsConsumed <= 300 THEN
          tier2Units <- unitsConsumed - 100
        ELSE
          tier2Units <- 200
          tier3Units <- unitsConsumed - 300
        ENDIF
      ENDIF
      tier1Cost <- tier1Units * 1.50
      tier2Cost <- tier2Units * 2.50
      tier3Cost <- tier3Units * 4.00
      baseBill  <- tier1Cost + tier2Cost + tier3Cost
      surcharge <- 0.0
      IF unitsConsumed > 300 THEN
        surcharge <- baseBill * 0.10
      ENDIF
      totalBill <- baseBill + surcharge
    ENDMETHOD

    METHOD DisplayBill()
      IF isValid = FALSE THEN
        DISPLAY "No bill: invalid meter data"
      ELSE
        DISPLAY "Consumer ID      : ", consumerID
        DISPLAY "Previous Reading : ", previousReading
        DISPLAY "Current Reading  : ", currentReading
        DISPLAY "Units Consumed   : ", unitsConsumed
        DISPLAY "Total Bill ($)   : ", totalBill
      ENDIF
    ENDMETHOD
ENDCLASS

BEGIN Main
  WaterConsumer consumer
  consumer.Initialize("WC-1001", 1450.0, 1820.0)
  consumer.CalculateConsumption()
  consumer.ComputeBill()
  consumer.DisplayBill()
END Main
```

Trace table: previousReading = 1450.0, currentReading = 1820.0

Step	Operation	Computation	Value
1	Validate	1450 >= 0, 1820 >= 0, 1820 >= 1450	isValid = TRUE
2	CalculateConsumption	1820.0 - 1450.0	unitsConsumed = 370.0
3	Tier decision	370 > 300, so use the third branch	tier1 = 100, tier2 = 200, tier3 = 70
4	Tier 1 cost	100 × 1.50	150.00
5	Tier 2 cost	200 × 2.50	500.00
6	Tier 3 cost	70 × 4.00	280.00
7	baseBill	150 + 500 + 280	930.00
8	Surcharge (units > 300)	930 × 10%	93.00
9	totalBill	930 + 93	**$1,023.00**

Question 2: Multi-Tier Tax & Precision Cast Engine

Topic: Control Structures (Selection), Data Types, Type Casting

Design overview

- Cap each deduction with selection (`IF d > cap THEN d <- cap`), add them, subtract from gross, and clamp the taxable income at zero.
- Progressive brackets are evaluated top-down with **IF / ELSE IF**, taxing only the slice of income inside each bracket (marginal, not flat, rates).
- Cess: raw tax × 1.04. Final amount converted REAL to INTEGER by adding 0.5 and truncating, i.e. `static_cast<long>(tax + 0.5)` = nearest-unit rounding.

Structured pseudo-code

```
BEGIN TaxCalculationEngine
  REAL    grossAnnualSalary, deduction80C, deductionHealthInsurance
  REAL    eligible80C, eligibleHealth, totalDeductions, taxableIncome
  REAL    rawTax, cess, totalTaxReal
  INTEGER finalTax
  CONSTANT REAL CAP_80C <- 150000.0
  CONSTANT REAL CAP_HEALTH <- 25000.0

  READ grossAnnualSalary, deduction80C, deductionHealthInsurance

  // Step 1: apply statutory caps
  IF deduction80C > CAP_80C THEN
    eligible80C <- CAP_80C
  ELSE
    eligible80C <- deduction80C
  ENDIF
  IF deductionHealthInsurance > CAP_HEALTH THEN
    eligibleHealth <- CAP_HEALTH
  ELSE
    eligibleHealth <- deductionHealthInsurance
  ENDIF

  // Step 2: taxable income, clamped at zero
  totalDeductions <- eligible80C + eligibleHealth
  taxableIncome   <- grossAnnualSalary - totalDeductions
  IF taxableIncome < 0 THEN
    taxableIncome <- 0.0
  ENDIF

  // Step 3: progressive slabs (marginal rates)
  IF taxableIncome <= 250000 THEN
    rawTax <- 0.0
  ELSE IF taxableIncome <= 500000 THEN
    rawTax <- (taxableIncome - 250000) * 0.05
  ELSE IF taxableIncome <= 1000000 THEN
    rawTax <- (250000 * 0.05) + (taxableIncome - 500000) * 0.20
  ELSE
    rawTax <- (250000 * 0.05) + (500000 * 0.20) + (taxableIncome - 1000000) * 0.30
  ENDIF

  // Step 4: Health & Education Cess of 4% on the raw tax
  cess         <- rawTax * 0.04
  totalTaxReal <- rawTax + cess

  // Step 5: explicit REAL -> INTEGER conversion (round to nearest)
  finalTax <- CAST_TO_INTEGER(totalTaxReal + 0.5)

  DISPLAY "Taxable Income: ", taxableIncome
```

```
  DISPLAY "Raw Tax: ", rawTax, "  Cess: ", cess
  DISPLAY "Final Tax Payable: ", finalTax
END TaxCalculationEngine
```

Trace table: gross = $1,200,000, 80C = $180,000, health insurance = $30,000

Step	Operation	Computation	Result
1	Cap 80C	180,000 > 150,000	eligible80C = 150,000
2	Cap health insurance	30,000 > 25,000	eligibleHealth = 25,000
3	totalDeductions	150,000 + 25,000	175,000
4	taxableIncome	1,200,000 - 175,000	1,025,000 (not negative, no clamp)
5	Slab test	1,025,000 > 1,000,000	top branch
6	Slab 2 part	250,000 × 5%	12,500
7	Slab 3 part	500,000 × 20%	100,000
8	Slab 4 part	(1,025,000 - 1,000,000) × 30% = 25,000 × 30%	7,500
9	rawTax	12,500 + 100,000 + 7,500	120,000.00
10	cess	120,000 × 4%	4,800.00
11	totalTaxReal	120,000 + 4,800	124,800.00
12	finalTax (cast)	CAST(124,800.00 + 0.5) = CAST(124,800.5)	**124,800**

Question 3: Bitwise Mask Validation & Sensor Health Telemetry

Topic: Operators in C++ (Bitwise &, |, ^, ~), Tokens, Constants

Design overview

- **Constants:** one named constant per bit; composite masks are built with bitwise OR so that no magic numbers appear in logic.
- **AND** isolates bits (`status AND mask`); **OR** builds composite masks; **XOR** with the mandatory mask exposes *missing* sensors; **NOT** clears the non-blocking radio bit so it never affects the verdict.
- No loops over bit positions: each bit is tested by its own mask in a separate IF.

Step 1: Hexadecimal bitmask constants

Constant	Hex	Binary	Meaning
GYRO_LOCKED	0x01	0000 0001	Bit 0, mandatory
ACCEL_CALIBRATED	0x02	0000 0010	Bit 1, mandatory
BARO_ONLINE	0x04	0000 0100	Bit 2, mandatory
GPS_FIX	0x08	0000 1000	Bit 3, mandatory
LOW_BATTERY	0x10	0001 0000	Bit 4, inhibitor
OVER_TEMP	0x20	0010 0000	Bit 5, inhibitor
RADIO_LINK	0x40	0100 0000	Bit 6, non-blocking
ACTUATOR_FAIL	0x80	1000 0000	Bit 7, inhibitor (fatal)
MANDATORY_MASK	0x0F	0000 1111	GYRO | ACCEL | BARO | GPS
INHIBITOR_MASK	0xB0	1011 0000	LOW_BATTERY | OVER_TEMP | ACTUATOR_FAIL

Step 2: Structured pseudo-code

```
CONSTANT BYTE GYRO_LOCKED      <- 0x01
CONSTANT BYTE ACCEL_CALIBRATED <- 0x02
CONSTANT BYTE BARO_ONLINE      <- 0x04
CONSTANT BYTE GPS_FIX          <- 0x08
CONSTANT BYTE LOW_BATTERY      <- 0x10
CONSTANT BYTE OVER_TEMP        <- 0x20
CONSTANT BYTE RADIO_LINK       <- 0x40
CONSTANT BYTE ACTUATOR_FAIL    <- 0x80
CONSTANT BYTE MANDATORY_MASK   <- GYRO_LOCKED OR ACCEL_CALIBRATED OR BARO_ONLINE OR GPS_FIX
CONSTANT BYTE INHIBITOR_MASK   <- LOW_BATTERY OR OVER_TEMP OR ACTUATOR_FAIL

FUNCTION CheckFlightReadiness(BYTE statusRegister) RETURNS BOOLEAN
  BYTE    effective, faults, present, missing
  BOOLEAN ready
  ready <- TRUE

  // Radio link is non-blocking: clear its bit so it can never influence the result
  effective <- statusRegister AND (NOT RADIO_LINK)

  // ---- Inhibitors: bits 4, 5 and 7 must all be 0 ----
  faults <- effective AND INHIBITOR_MASK
  IF faults != 0 THEN
    ready <- FALSE
    IF (faults AND LOW_BATTERY) != 0 THEN
      DISPLAY "FAULT: Bit 4 - Low Battery Warning"
    ENDIF
    IF (faults AND OVER_TEMP) != 0 THEN
      DISPLAY "FAULT: Bit 5 - Over-Temperature Warning"
    ENDIF
    IF (faults AND ACTUATOR_FAIL) != 0 THEN
      DISPLAY "FAULT: Bit 7 - Fatal Actuator Hardware Failure"
    ENDIF
  ENDIF

  // ---- Mandatory: bits 0-3 must all be 1 ----
  present <- effective AND MANDATORY_MASK
  missing <- present XOR MANDATORY_MASK          // 1-bits mark sensors that are NOT set
  IF missing != 0 THEN
    ready <- FALSE
    IF (missing AND GYRO_LOCKED) != 0 THEN
      DISPLAY "MISSING: Bit 0 - Gyroscope not locked"
    ENDIF
    IF (missing AND ACCEL_CALIBRATED) != 0 THEN
      DISPLAY "MISSING: Bit 1 - Accelerometer not calibrated"
    ENDIF
    IF (missing AND BARO_ONLINE) != 0 THEN
      DISPLAY "MISSING: Bit 2 - Barometric altimeter offline"
    ENDIF
```

```
    IF (missing AND GPS_FIX) != 0 THEN
      DISPLAY "MISSING: Bit 3 - GPS fix not acquired"
    ENDIF
  ENDIF

  IF ready = TRUE THEN
    DISPLAY "FLIGHT READY"
  ELSE
    DISPLAY "NOT READY FOR FLIGHT"
  ENDIF
  RETURN ready
END FUNCTION
```

Step 3: Trace for the two test bytes

Operation	Test 1: 0x4F (nominal)	Test 2: 0x3B (low battery + missing sensor)
statusRegister (binary)	0x4F = 0100 1111	0x3B = 0011 1011
effective = status AND (NOT 0x40) [NOT 0x40 = 0xBF]	0x4F AND 0xBF = 0x0F (radio bit cleared)	0x3B AND 0xBF = 0x3B (bit 6 already 0)
faults = effective AND 0xB0	0x0F AND 0xB0 = 0x00	0x3B AND 0xB0 = 0x30 (bits 4 and 5)
Inhibitor test	faults = 0: no fault messages	(0x30 AND 0x10) != 0: **FAULT bit 4 Low Battery** (0x30 AND 0x20) != 0: **FAULT bit 5 Over-Temperature** (0x30 AND 0x80) = 0: no actuator fault
present = effective AND 0x0F	0x0F	0x3B AND 0x0F = 0x0B (bits 0, 1, 3)
missing = present XOR 0x0F	0x0F XOR 0x0F = 0x00	0x0B XOR 0x0F = 0x04 (bit 2)
Mandatory test	missing = 0: nothing missing	**MISSING bit 2: Barometric altimeter offline**
Result	**FLIGHT READY** (radio link active is ignored)	**NOT READY FOR FLIGHT** (returns FALSE)

Question 4: Optimized Prime Factorization & Divisor Engine

Topic: Control Structures (Iteration: `for`/`while`, Jump: `break`, `continue`, `goto`)

Design overview

- **Input validation by jump:** a label `READ_INPUT` with `GOTO READ_INPUT` repeats the prompt while N <= 0.
- **O(sqrt N) divisor search:** divisors come in pairs (d, N/d), so only d up to floor(sqrt N) is scanned. The loop test `d * d <= N` avoids a floating-point square root.
- **CONTINUE** skips non-divisors. When d divides N, add both d and N/d, but add only once when d = N/d (perfect square).
- **Sum starts at 1** (1 divides every N > 1 and N itself is excluded). N = 1 is handled explicitly: it has no proper divisor other than itself, so the sum is 0.

Structured pseudo-code

```
BEGIN DivisorAndPrimeEngine
  INTEGER N, d, pair, sumOfDivisors

  READ_INPUT:
    DISPLAY "Enter a positive integer N: "
    READ N
    IF N <= 0 THEN
      DISPLAY "Error: N must be positive. Try again."
      GOTO READ_INPUT
    ENDIF

  IF N = 1 THEN                          // edge case handled explicitly
    sumOfDivisors <- 0
  ELSE
    sumOfDivisors <- 1                   // 1 is a proper divisor of every N > 1
    d <- 2
    WHILE d * d <= N DO                  // d runs from 2 up to floor(sqrt(N))
      IF N MOD d != 0 THEN
        d <- d + 1
        CONTINUE                         // not a divisor: skip to next candidate
      ENDIF
      sumOfDivisors <- sumOfDivisors + d
      pair <- N DIV d
      IF pair != d THEN                  // avoid double counting when d = sqrt(N)
        sumOfDivisors <- sumOfDivisors + pair
      ENDIF
      d <- d + 1
    ENDWHILE
  ENDIF

  DISPLAY "Sum of proper divisors = ", sumOfDivisors
  IF sumOfDivisors = N THEN
    DISPLAY N, " is a PERFECT number"
  ELSE IF sumOfDivisors > N THEN
    DISPLAY N, " is an ABUNDANT number"
  ELSE
    DISPLAY N, " is a DEFICIENT number"
  ENDIF
END DivisorAndPrimeEngine
```

Trace table A: N = 28 (floor(sqrt 28) = 5, so d = 2..5; initial sum = 1)

d	d*d <= 28 ?	28 MOD d	Action	N / d	sumOfDivisors
-	-	-	initialise	-	1
2	4 <= 28 yes	0	divisor: add 2 and pair	14	1 + 2 + 14 = 17
3	9 <= 28 yes	1	CONTINUE (skip)	-	17
4	16 <= 28 yes	0	divisor: add 4 and pair	7	17 + 4 + 7 = 28
5	25 <= 28 yes	3	CONTINUE (skip)	-	28
6	36 <= 28 no	-	loop ends	-	28

Sum = 28 = N, so 28 is a **PERFECT number** (proper divisors 1, 2, 4, 7, 14).

Trace table B: N = 12 (floor(sqrt 12) = 3, so d = 2..3; initial sum = 1)

d	d*d <= 12 ?	12 MOD d	Action	N / d	sumOfDivisors
-	-	-	initialise	-	1
2	4 <= 12 yes	0	divisor: add 2 and pair	6	1 + 2 + 6 = 9
3	9 <= 12 yes	0	divisor: add 3 and pair (3 != 4)	4	9 + 3 + 4 = 16
4	16 <= 12 no	-	loop ends	-	16

Sum = 16 > 12, so 12 is an **ABUNDANT number** (proper divisors 1, 2, 3, 4, 6).

Question 5: State-Machine ATM Controller with Sentinel Loops

Topic: Control Structures (`switch-case`, `while`, nested `if-else`), Data Types

Design overview

- **Two states:** AUTHENTICATING then SESSION. An attempt counter limits PIN entry to 3 tries; the 3rd failure locks the card and terminates.
- **Sentinel-controlled loop:** the session loop runs `WHILE choice != 4`; option 4 (Exit) is the sentinel.
- **Menu dispatch** with CASE (switch). Withdrawal uses nested IFs: amount > 0, then amount <= balance, then `amount MOD 100 = 0`.
- **Data types:** PIN, choice, attempts are INTEGER; balance and amounts are REAL; flags are BOOLEAN.

Structured pseudo-code

```
BEGIN ATM_SessionController
  CONSTANT INTEGER CORRECT_PIN   <- 4821
  CONSTANT INTEGER MAX_ATTEMPTS  <- 3
  INTEGER enteredPIN, attempts, choice
  REAL    balance, amount
  BOOLEAN authenticated

  balance       <- <account balance loaded from the bank>
  attempts      <- 0
  authenticated <- FALSE

  // ---------- Phase 1: authentication (max 3 attempts) ----------
  WHILE attempts < MAX_ATTEMPTS AND authenticated = FALSE DO
    DISPLAY "Enter 4-digit PIN: "
    READ enteredPIN
    IF enteredPIN = CORRECT_PIN THEN
      authenticated <- TRUE
    ELSE
      attempts <- attempts + 1
      IF attempts < MAX_ATTEMPTS THEN
        DISPLAY "Incorrect PIN. Attempts left: ", MAX_ATTEMPTS - attempts
      ENDIF
    ENDIF
  ENDWHILE

  IF authenticated = FALSE THEN
    DISPLAY "Card locked after 3 incorrect attempts."
    STOP                                  // terminate immediately
  ENDIF

  // ---------- Phase 2: sentinel-controlled session loop ----------
  choice <- 0
  WHILE choice != 4 DO
    DISPLAY "1: Balance  2: Deposit  3: Withdraw  4: Exit"
    READ choice
    CASE choice OF
      1:
        DISPLAY "Available balance: ", balance
      2:
        DISPLAY "Enter deposit amount: "
        READ amount
        IF amount > 0 THEN
          balance <- balance + amount
          DISPLAY "Deposit successful. New balance: ", balance
        ELSE
          DISPLAY "Invalid amount. Deposit must be positive."
        ENDIF
      3:
```

```
        DISPLAY "Enter withdrawal amount: "
        READ amount
        IF amount > 0 AND amount <= balance THEN
          IF amount MOD 100 = 0 THEN
            balance <- balance - amount
            DISPLAY "Please collect cash. New balance: ", balance
          ELSE
            DISPLAY "Declined: amount must be a multiple of 100."
          ENDIF
        ELSE
          DISPLAY "Declined: invalid amount or insufficient funds."
        ENDIF
      4:
        DISPLAY "Thank you. Session ended."
      DEFAULT:
        DISPLAY "Invalid option. Please choose 1 to 4."
    ENDCASE
  ENDWHILE
END ATM_SessionController
```

Dry-run trace (assumed opening balance = $5,000.00)

The paper does not state the starting balance, so $5,000 is assumed purely to make the trace concrete.

Step	Input	Condition checked	Action / output	attempts	balance
1	PIN = 1234	1234 = 4821? No	attempts = 1; "Incorrect PIN. Attempts left: 2"	1	5000
2	PIN = 4821	4821 = 4821? Yes	authenticated = TRUE; authentication loop exits	1	5000
3	choice = 3	CASE 3 (withdraw)	prompt for amount	-	5000
4	amount = 250	250 > 0 and 250 <= 5000: yes. 250 MOD 100 = 50, not 0	"Declined: amount must be a multiple of 100." (balance unchanged)	-	5000
5	choice = 3	CASE 3 (withdraw)	prompt for amount	-	5000
6	amount = 300	300 > 0 and 300 <= 5000: yes. 300 MOD 100 = 0	balance = 5000 - 300; "Please collect cash. New balance: 4700"	-	**4700**
7	choice = 4	sentinel: choice != 4 is false	"Thank you. Session ended." Loop exits	-	4700